<ALGORITMA PEMROGRAMAN 1B>

PERTEMUAN KE 6 FUNGSI PADA PYTHON

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Cangkupan Materi Pertemuan 6

1

Modularitas dan Fungsi pada python

- Fungsi Aritmatika
- Fungsi Rekursif
- Fungsi Matematika

2

Pembuatan Fungsi Baru

3

Pemanggilan Fungsi

- Scope Variabel : Menggunakan parameter variabel local dalam fungsi
- Nilai Default Fungsi

1

Modularitas dan Fungsi pada python

ALGORITMA PEMROGRAMAN 1B

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Modularitas

- Membagi program menjadi bagian-bagian kecil (modul) yang masing-masing punya tugas tertentu
- Contohnya dalam kepengurusan OSIS, terdapat berbagai divisi yang memiliki tugas masing-masing
- Dalam pemrograman, kalian dapat membuat fungsi-fungsi kecil (masing-masing fungsi memiliki tugas tertentu), lalu digabungkan dan dapat membentuk struktur program yanglengkap.

CC BY NC SA

Keuntungan Modularitas

- Program lebih mudah dipahami (abstraksi)
- Lebih mudah untuk diperbaiki
- Dapat digunakan untuk program lain
- Lebih mudah untuk pembagian tugas kerja (dekomposisi)

BY NC SA

Fungsi pada Python

- Fungsi (Function) adalah suatu program terpisah dalam blok sendiri yang berfungsi sebagai sub-program (modul program) yang merupakan sebuah program kecil untuk memproses sebagian dari pekerjaan program utama.
- Fungsi adalah grup/blok program untuk melakukan tugas tertentu yang berulang.
- Fungsi membuat kode program menjadi reusable, artinya hanya di definisikan sekali saja, dan kemudian bisa digunakan berulang kali dari tempat lain di dalam program.

Fungsi pada Python

- Fungsi memecah keseluruhan program menjadi bagian – bagian yang lebih kecil.
- Semakin besarnya program, maka fungsi akan membuatnya menjadi lebih mudah diorganisir dan dimanage.
- Sejauh ini, sudah menggunakan beberapa fungsi, misalnya fungsi print(), type(), dan sebagainya. Fungsi tersebut adalah fungsi bawaan dari Python. Fungsi dapat dibuat sesuai kebutuhan

Fungsi pada Python

Keuntungan menggunakan fungsi :

1. Program besar dapat di pisah-pisah menjadi program-program kecil melalui function.
2. Kemudahan dalam mencari kesalahan-kesalahan karena alur logika jelas dan kesalahan dapat dilokalisasi dalam suatu modul tertentu.
3. Memperbaiki atau memodifikasi program dapat dilakukan pada suatu modul tertentu saja tanpa menggangu keseluruhan program.
4. Dapat digunakan kembali (Reusability) oleh program atau fungsi lain.
5. Meminimalkan penulisan perintah yang sama.

Fungsi pada Python - Kategori Fungsi

1.Standard Library Function

adalah fungsi-fungsi yang telah disediakan oleh Interpreter Python dalam file-file atau librarynya.

Misalnya: raw_input(), input(), print(), open(), len(), max(), min(), abs() dll.

2.Programme-Defined Function

Adalah function yang dibuat oleh programmer sendiri.

Function ini memiliki nama tertentu yang unik dalam program, letaknya terpisah dari program utama, dan bisa dijadikan satu ke dalam suatu library buatan programmer itu sendiri.

Fungsi pada Python - Kategori Fungsi

Dalam python terdapat dua perintah yang dapat digunakan untuk membuat sebuah fungsi, yaitu def dan lambda.

def adalah perintah standar dalam python untuk mendefinisikan sebuah fungsi. Tidak seperti function dalam bahasa pemrograman compiler seperti C/C++, def dalam python merupakan perintah yang executable, artinya function tidak akan aktif sampai python me- running perintah def tersebut.

lambda, dalam python lebih dikenal dengan nama Anonymous Function (Fungsi yang tidak disebutkan namanya). Lambda bukanlah sebuah perintah (statemen) namun lebih kepada ekspresi (expression).

Fungsi pada Python - Fungsi Aritmatika

Fungsi Aritmatika – Adalah fungsi matematika sederhana yang terdiri dari penjumlahan, pengurangan, perkalian, pembagian dan gabungan Perhitungan dalam aritmatika dilakukan menurut suatu urutan operasi yang menentukan operasi aritmatika mana yang lebih dulu akan dilakukan

Program fungsi aritmatika

```
def tambah(a,b):
    c = a+b
    print(a, " + ", b, " = ", c)

tambah(5,10)
```

output:

```
5  +  10  =  15
```

Fungsi pada Python - Fungsi Rekursif

Fungsi Rekursif merupakan suatu fungsi yang memanggil dirinya sendiri. Artinya, fungsi tersebut dipanggil di dalam tubuh fungsi itu sendiri.

Tujuan di lakukan rekursif adalah untuk menyederhanakan penulisan program dan menggantikan bentuk iterasi. Dengan rekursi, program akan lebih mudah dilihat.

Mencari nilai faktorial dari suatu bilangan bulat positif adalah salah satu pokok bahasan yang memudahkan pemahaman mengenai fungsi rekursif.

Fungsi pada Python - Fungsi Rekursif

Berikut adalah fungsi faktorial yang diselesaikan dengan cara biasa :

Konsep faktorial,

N ! = faktorial(N) = 1 * 2 * 3 … * N

Dalam pemrograman konsep dari faktorial seperti berikut,

faktorial(N)= N!

= N * (N-1)!

= N * (N-1) * (N-2)!

= N * (N-1) * (N-2) … * 3 * 2 * 1

Fungsi pada Python - Fungsi Rekursif

Program mencari nilai faktorial :

```
# Fungsi Rekursif faktorial
def faktorial(nilai):
    if nilai <= 1:
        return 1
else:
    return nilai * faktorial(nilai - 1)
#Program utama
for i in range(11):
    print "%2d ! = %d" % (i, faktorial(i))
```

Output	:		
0	!	=	1
1	!	=	1
2	!	=	2
3	!	=	6
4	!	=	24
5	!	=	120
6	!	=	720
7	!	=	5040
8	!	=	40320
9	!	=	362880
10	!	=	3628800

Fungsi Matematika

- Python menyediakan beberapa fungsi matematika dasar yang dapat langsung digunakan di kode program kalian
- Python juga menyediakan library tambahan, math, untuk fungsi matematika yang lebih berkelanjutan.

BY NC SA

Fungsi Bawaan

Fungsi	Kegunaan	Contoh	Hasil
`min()`	Mencari nilai terkecil	`x = min(7, 23, 13)`	7
`max()`	Mencari nilai terbesar	`x = max(7, 23, 13)`	23
`abs()`	Mengembalikan nilai absolut (positif)	`x = abs(-1.3)`	1.3
`pow()`	Mengembalikan nilai pangkat	`x = pow(2, 5)`	32

CC BY NC SA

Fungsi Tambahan

Fungsi	Kegunaan	Contoh	Hasil
`math.sqrt()`	Mengembalikan nilai akar kuadrat	`x = math.sqrt(64)`	`8.0`
`math.ceil()`	Membulatkan ke atas bilang bulat terdekat	`x = math.ceil(1.3)`	`2`
`math.floor()`	Membulatkan ke bawah bilangan bulat terdekat	`x = math.floor(1.3)`	`1`
`math.pi`	Mengembalikan nilai PI	`x = math.pi`	`3.141592653589793`

CC BY NC SA

2

Pembuatan Fungsi

ALGORITMA PEMROGRAMAN 1B

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Pembuatan Fungsi pada Python - def

Statemen **def** digunakan untuk mendeklarasikan fungsi. Sedangkan statemen return digunakan untuk mengembalikan suatu nilai kepada bagian program yang memanggil fungsi. Bentuk umum untuk mendeklarasikan fungsi adalah sebagai berikut :

```
def <nama_fungsi>(arg1, arg2, arg3, ...,argN) :
    <statemen-statemen>
```

Contoh:

```
def perkenalan(nama):
    print( "halo, nama saya adalah" , nama)
```

Pembuatan Fungsi pada Python - def

Sebuah fungsi diawali dengan statemen def kemudian diikuti oleh sebuah nama_fungsi nya. Sebuah fungsi dapat memiliki daftar argumen (parameter) ataupun tidak. Tanda titik dua (:) menandakan awal pendefinisian tubuh dari fungsi yang terdiri dari statemen-statemen.

Tubuh fungsi yang memiliki statemen return :

```
def <nama_fungsi>(arg1, arg2, arg3, ...,argN) :
...    <statemen-statemen>
...    return <value>
```

Pembuatan Fungsi pada Python - def

Contoh:

```
def perkalian(a,b):
    c =  a*b
    return c
# Program Utama
print(perkalian(5,10))
```

output:

50

Statemen return dapat diletakkan di bagian mana saja dalam tubuh fungsi. Statemen return menandakan akhir dar pemanggilan fungsi dan akan mengirimkan suatu nilai balik kepada program yang memanggil fungsi tersebut. Statemen return bersifat opsional, artinya jika sebuah fungsi tidak memiliki statemen return, maka sebuah fungsi tidak akan mengembalikan suatu nilai apapun.

3

Pemanggilan Fungsi

ALGORITMA PEMROGRAMAN 1B

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Pemanggilan Fungsi pada Python

Pernyataan def mendefinisikan sebuah fungsi dengan nama ucapan. Fungsi ucapan tidak memiliki daftar argumen dan tidak meminta nilai kembalian.

Pendefinisian fungsi ucapan diakhiri dengan tanda (:), kemudian diikuti oleh statemen print yang menjadi isi dari tubuh fungsi.

Lalu untuk memanggil fungsi ucapan() kita gunakan perintah, bentuk umum manggil fungsi adalah:

```
<nama_fungsi>()
```

contoh:

```
ucapan()
```

Pemanggilan Fungsi pada Python

Contoh pemanggilan fungsi :

```
>>> def ucapan():
...  print "Anda sedang menggunakan fungsi"
...
>>> ucapan()
Anda sedang menggunakan fungsi
```

Pemanggilan Fungsi pada Python- Scope Variabel

Scope variabel atau cakupan variabel merupakan suatu keadaan dimana pendeklarasian sebuah variabel di tentukan. Dalam scope variabel dikenal dua istilah yaitu **local** dan **global** .

Variabel disebut **local** ketika variabel tersebut didefinisikan didalam sebuah fungsi (def). Artinya, variabel tersebut hanya dapat di gunakan dalam cakupan fungsi tersebut saja.

Dan jika sebuah variabel didefinisikan diluar fungsi maka variabel tersebut bersifat **global**. Artinya, variabel tersebut dapat digunakan oleh fungsi lain atau pun program utamanya.

Pemanggilan Fungsi pada Python- Scope Variabel

Contoh penggunaan scope variabel :

```
def contohScope(X):
    X = 10
    print("Nilai X di dalam fungsi, x = ", X)

# program utama
X = 30
print("Nilai x di luar fungsi, x = ", X)
contohScope(X)
```

Output :

```
Nilai x di luar fungsi, x = 30
Nilai X di dalam fungsi, x = 10
```

Pada contoh disamping, variabel X didefinisikan di dua tempat yaitu di dalam fungsi contohScope() dan di dalam program utama.

Ketika nilai X awal di beri nilai 30, kemudian di cetak, nilai X masih bernilai 30. Namun ketika kita memanggil fungsi contohScope() dengan mengirim parameter X yang bernilai 30, terlihat bahwa nilai X yang berlaku adalah nilai X yang didefinisikan didalam fungsi tersebut.

Atau nilai X yang bernilai 10. ini terbukti bahwa variabel X yang di cetak dalam fungsi contohScope() merupakan variabel local yang didefinisikan didalam fungsi, bukan variabel X global yang dicetak di luar fungsi.

Nilai Bawaan (Default) Paramater

Nilai Bawaan

- Fungsi dengan parameter juga bisa memiliki nilai bawaan (default parameter).
- Apabila kalian mendefinisikan nilai bawaan parameter, maka kalian bisa memanggil fungsi tanpa memberikan nilai khusus.
- Jika kalian memberikan nilai tertentu pada parameter, maka yang akan digunakan adalah nilai baru tersebut.
- Jika kalian tidak memberikan nilai tertentu, maka yang akan digunakan adalah nilai bawaan.

CC BY NC SA

Contoh dalam kehidupan sehari-hari

Bayangkan kalian ingin memberi es kopi susu di kedai kopi terdekat:

- Kalau kalian tidak menyebutkan detail informasi lain, maka penjual otomatis memberi yang biasanya (default, misalnya takaran gula cair 30 ml).
- Kalau kalian menyebutkan "*jangan terlalu manis*", maka penjual akan mengurangi takaran gula sesuai yang kalian minta, bukan default.

CC BY NC SA

Penggunaan Nilai Bawaan

```
def nama_fungsi(parameter1, parameter2="nilai_bawaan"):
```

Parameter biasa

Parameter Bawaan/Default

```
nama_fungsi(isi_parameter1)
```

Digunakan untuk penggunaan parameter default

```
nama_fungsi(isi_parameter1, isi_paramater2)
```

Digunakan untuk penggunaan Seluruh parameter

CC BY NC SA

Contoh Program

Default Value

```
def sapa(nama, pesan="Selamat Pagi"):
    print(pesan, nama)
```

```
sapa("Ani")
```

```
Selamat Pagi Ani
```

Karena terdapat default Value, Sehingga cukup **satu parameter** saja yang dikirim

```
sapa("Budi", "Halo")
```

```
Halo Budi
```

Karena **dikirim dua parameter**, maka isi parameter pesan berisi inputian dari fungsi bukan dari default Value

CC BY NC SA

Latihan 1

Buatkan dalam 1 kode program utuh dimana program menggunakan fungsi dapat menghitung keliling dan luas lingkaran dengan parameter jari-jari merupakan hasil masukan (input)!

Latihan 2

Buatkan program menggunakan fungsi untuk menu kalkulator sederhana dengan inputan nilai 1 dan nilai 2, untuk perhitungan :

1. Penjumlahan
2. Pengurangan
3. Perkalian
4. Pembagian
5. Hasil Sisa Bagi

Latihan 3

- Buatlah sebuah program yang akan menghitung harga akhir sesudah pajak. Di mana harga akhir adalah harga dasar ditambah biaya pajaknya. Besaran pajak bawaan (default) adalah 10% dari harga dasar.
- Jika pengguna tidak memasukkan besaran pajak khusus, maka pajak yang digunakan sesuai nilai bawaan, yaitu 10%. Jika pengguna memasukkan besaran pajak khusus, misal 5, maka pajak yang digunakan adalah 5%.

Contoh program:

```
Harga dasar: 10000 dan pajak normal (10%)
Harga sesudah pajak normal: 11000.0

Harga dasar: 10000 dan pajak khusus (5%)
Harga sesudah pajak khusus: 10500.0
```

Solusi Kasus

```
def hitung_pajak(besaran, pajak=0.10):
  harga_setelah_pajak = besaran + (besaran * pajak)

  return harga_setelah_pajak
```

Fungsi dengan Parameter bawaan

```
harga_10persen = hitung_pajak(10000)
print(harga_10persen)
```

```
11000.0
```

Memanggil Fungsi dengan Parameter bawaan

```
harga_5persen = hitung_pajak(10000, 0.05)
print(harga_5persen)
```

```
10500.0
```

Memanggil Fungsi seperti biasa

CC BY NC SA

Daftar Pustaka

- http://dini3asa.staff.gunadarma.ac.id/Downloads/files/19688/FUNGSI.pdf
  - https://drive.google.com/drive/folders/1XxWPQJzdiuaq_nX18uglGAPx5nLUlchJ?usp=sharing
    - Guido van Rossum. 2003. Python Tutorial. PythonLabs
    - Bob Dowling. An introduction to Python for absolute beginners. University Computing Service
    - Mark Lutz. 2009. Powerfull Object Oriented Programming : Learning Python 4th Edition. O' Reily.
    - Referensi tambahan :
    - Dave Kuhlman. 2013. A Python Book: Beginning Python, Advanced Python, and Python Exercises.

TERIMA KASIH

PROGRAM STUDI INFORMATIKA
UNIVERSITAS GUNADARMA

Tim Penyusun:
1.Dini Triasanti
2.
3